

Scanno Integrated AI Analyzer API Documentation

This document provides a detailed overview of the RESTful API endpoints, data schemas, and authentication flows for the Scanno-auth backend service.

1. Authentication & Roles

The API uses **Bearer Token (JWT)** authentication via the `Authorization: Bearer <token>` header for most endpoints.

Roles

Role	Access	Purpose
Engineer (<code>ROLE_ENGINEER</code>)	Access to Chat Core, User Registration, Login, and History.	The primary user role for interacting with the AI analysis features.
Admin (<code>ROLE_ADMIN</code>)	Access to Admin Login and API Key management endpoints.	Responsible for setting up and managing the internal OpenAI API key.

2. User Authentication Endpoints (`/user`)

These endpoints are used by **Engineer** users for registration, login, and token management.

POST `/user/register`

Registers a new Engineer user.

- **Tag:** User Authentication
- **Authentication:** None
- **Request Body (Schema:** `UserCreate` **)**

```
{
  "email": "engineer@company.com",
  "password": "securepassword123"
}
```

- **Responses:** | Status | Description | | :--- | :--- | | 200 OK | User created successfully. | | 400 Bad Request | Email already registered. |

POST `/user/login`

Authenticates the Engineer and returns access and refresh tokens.

- **Tag:** User Authentication
- **Authentication:** None
- **Request Body (Schema:** `UserLogin` **)**

```
{
  "email": "engineer@company.com",
  "password": "securepassword123"
}
```

- **Response Body (Schema: Token)**

```
{
  "access_token": "...",
  "refresh_token": "...",
  "token_type": "bearer"
}
```

- **Token Expiry:**

- access_token : ~30 minutes
- refresh_token : ~7 days

POST /user/change-password

Allows a logged-in Engineer to update their password.

- **Tag:** User Authentication
- **Authentication:** Required (Engineer access token)
- **Request Body (Schema: PasswordChange)**

```
{
  "old_password": "securepassword123",
  "new_password": "newsecurepassword456"
}
```

- **Responses:** | Status | Description | | :--- | :--- | | 200 OK | Password updated successfully. | | 401 Unauthorized | Invalid credentials (token or old password). |

3. AI Core Chat Endpoints

These are the primary endpoints for analysis and chat, requiring **Engineer** authentication.

POST /analyze-report

Initiates a new AI analysis session by uploading a file. This is the **starting point** for any new conversation.

- **Tag:** AI Core Chat
- **Authentication:** Required (Engineer access token)
- **Request:**
 - **Form Data:** file (UploadFile)
 - **Supported Types:** PDF, JPG, PNG

- **Response Body (Schema:** `AnalysisResponse` **)**

```
{
  "session_id": "8489582d-1579-4d2d-8b02-a7d03a5e8888",
  "file": "Uploaded file name",
  "report": {
    "summary": "Initial summary from the AI model."
  }
}
```

- **Flow Notes:**

- The backend processes the file, initiates the AI analysis, and saves the initial chat transcript to **Redis**.
- The `session_id` returned **MUST** be used for all subsequent chat interactions.

POST /chat

Continues an existing AI analysis session.

- **Tag:** AI Core Chat
- **Authentication:** Required (Engineer access token)
- **Request Body (Schema:** `ChatRequest` **)**

```
{
  "session_id": "8489582d-1579-4d2d-8b02-a7d03a5e8888",
  "message": "What is the key conclusion of this report?"
}
```

- **Response Body (String)**

- Returns a plain text response from the AI model.

- **Flow Notes:**

- The backend loads the history from Redis using `session_id`, appends the new message, calls the AI, appends the AI's response, and saves the updated history back to Redis (resetting the 1-hour session TTL).

GET /history

Retrieves the permanent log of analysis summaries (not the full chat) for the current Engineer.

- **Tag:** AI Core Chat
- **Authentication:** Required (Engineer access token)
- **Response Body (List of Schema:** `HistoryResponse` **)**

```
[
  {
    "id": 1,
    "engineer_email": "engineer@company.com",
    "chat_data": "Initial summary analysis for Report Q3-2024.",
  }
]
```

```

    "timestamp": "2025-11-06T10:00:00.000Z"
  }
]

```

DELETE /history

Deletes all permanent history logs for the current Engineer.

- **Tag:** AI Core Chat
- **Authentication:** Required (Engineer access token)
- **Responses:** | Status | Description | | :--- | :--- | | 200 OK | Successfully deleted X history entries. |

GET /session/{session_id}

Debugging/Display Endpoint: Retrieves the complete, turn-by-turn conversation history for a specific session ID from Redis.

- **Tag:** AI Core Chat
- **Authentication:** Required (Engineer access token)
- **Path Parameter:** session_id (string)
- **Response Body (Schema:** FullSessionHistory)

```

{
  "session_id": "8489582d-1579-4d2d-8b02-a7d03a5e8888",
  "messages": [
    {"role": "user", "content": "Analyze this report..."},
    {"role": "assistant", "content": "Here is the summary..."},
    // ... more messages
  ]
}

```

4. Admin Endpoints (/admin)

These endpoints are for administrative use only and require **Admin** authentication.

POST /admin/login

Authenticates the Admin and returns access and refresh tokens.

- **Tag:** Admin (Key Management)
- **Authentication:** None
- **Request Body (Schema:** UserLogin)
 - Use the ADMIN_EMAIL and ADMIN_PASSWORD defined in the server's environment configuration.
- **Response Body (Schema:** Token)
 - Same format as /user/login .

POST /admin/apikey

Sets or updates the internal OpenAI API key.

- **Tag:** Admin (Key Management)
- **Authentication:** Required (Admin access token)
- **Request Body (Schema:** `APIKeyCreate`)

```
{
  "api_key": "sk-XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
}
```

- **Responses:** | Status | Description | | :--- | :--- | | 200 OK | API Key saved successfully. | | 400 Bad Request | Invalid key format. |

GET /admin/apikey

Checks the configuration status of the OpenAI API key.

- **Tag:** Admin (Key Management)
- **Authentication:** Required (Admin access token)
- **Response Body:**

```
{
  "status": "Configured" or "Not set",
  "key_preview": "sk-XXXX..." or null
}
```

5. Pydantic Schemas Reference

Schema	Purpose	Used in Endpoints	Fields
UserCreate	For Engineer registration.	POST /user/register	email (EmailStr), password (str)
UserLogin	For user/admin login.	POST /user/login , POST /admin/login	email (EmailStr), password (str)
Token	JWT access and refresh tokens.	POST /user/login , POST /admin/login	access_token (str), refresh_token (str), token_type (str)
HistoryResponse	Permanent database record of analysis.	GET /history	id (int), engineer_email (str), chat_data (str), timestamp (datetime)
ChatRequest	Sending a message to an active chat.	POST /chat	session_id (str), message (str)

AnalysisResponse	Response upon file upload/initial analysis.	POST /analyze-report	session_id (str), file (str), report (dict)
FullSessionHistory	Full transcript retrieval.	GET /session/{session_id}	session_id (str), messages (List[ChatMessage])